Homework-8

1.Simulate the BFS and DFS traversals for the given graph.

```
#include<stdio.h>
#define MAX 100
int g[MAX][MAX],visited[MAX],n;
void dfs(int v){
int i;
visited[v]=1;
printf("%d ",v);
for(i=0;i<n;i++)
if(g[v][i]&&!visited[i])dfs(i);
}
void bfs(int start){
int q[MAX],front=0,rear=0,i;
for(i=0;i<n;i++)visited[i]=0;
visited[start]=1;
q[rear++]=start;
while(front<rear){
int v=q[front++];
printf("%d ",v);
for(i=0;i<n;i++){
if(g[v][i]&&!visited[i]){
visited[i]=1;
q[rear++]=i;
}
}
```

```
}
}
int main(){
int i,j;
scanf("%d",&n);
for(i=0;i<n;i++)
for(j=0;j<n;j++)
scanf("%d",&g[i][j]);
printf("BFS: ");
bfs(0);
for(i=0;i<n;i++)visited[i]=0;
printf("\nDFS: ");
dfs(0);
return 0;
}
```

OUTPUT:

BFS: 0 1 2 3

DFS: 0 1 3 2

2.Find the minimum spanning tree for the given graph using prim's and kruskal's algorithm

(i)Prim's algorithm.

```
#include<stdio.h>
#define MAX 100
#define INF 999999
int main(){
int g[MAX][MAX],vis[MAX]={0},n,i,j,u,v,min,cost=0;
scanf("%d",&n);
```

```
for(i=0;i<n;i++)
for(j=0;j<n;j++){
scanf("%d",&g[i][j]);
if(i!=j&&g[i][j]==0)g[i][j]=INF;
}
vis[0]=1;
for(i=0;i<n-1;i++){
min=INF;
u=-1;
for(j=0;j<n;j++){
if(vis[j]){
for(v=0;v<n;v++){
if(!vis[v]&&g[j][v]<min){
min=g[j][v];
u=j;
}
}
}
}
if(u==-1){printf("No spanning tree");return 0;}
for(v=0;v<n;v++)
if(!vis[v]&&g[u][v]==min)break;
printf("%d-%d:%d\n",u,v,min);
vis[v]=1;
cost+=min;
}
```

```
printf("Minimum cost=%d\n",cost);
return 0;
}
```

(ii)Kruskal's algorithm.

```
#include<stdio.h>
#define MAX 100
struct edge{
int u,v,w;
};
struct edge e[MAX],t;
int parent[MAX];
int find(int x){
if(parent[x]!=x)parent[x]=find(parent[x]);
return parent[x];
}
int main(){
int n,m,i,j,a,b,count=0,cost=0;
scanf("%d%d",&n,&m);
for(i=0;i<m;i++)scanf("%d%d%d",&e[i].u,&e[i].v,&e[i].w);
for(i=0;i<n;i++)parent[i]=i;
for(i=0;i<m-1;i++)
for(j=0;j<m-i-1;j++)
if(e[j].w>e[j+1].w){
t=e[j];e[j]=e[j+1];e[j+1]=t;
}
for(i=0;i<m;i++){
```

```
a=find(e[i].u);
b=find(e[i].v);
if(a!=b){
parent[a]=b;
printf("%d-%d:%d\n",e[i].u,e[i].v,e[i].w);
cost+=e[i].w;
count++;
}
}
if(count!=n-1)printf("No spanning tree\n");
else printf("Minimum cost=%d\n",cost);
return 0;
}
```

OUTPUT:

```
2-3:4
0-3:5
0-1:10
Minimum cost=19
```

3.Find the minimum spanning tree.

```
#include<stdio.h>
#define MAX 100
struct edge{
int u,v,w;
};
struct edge e[MAX],temp;
int parent[MAX];
```

```
int find(int x){
if(parent[x]!=x)parent[x]=find(parent[x]);
return parent[x];
}
int main(){
int n,m,i,j,a,b,count=0,cost=0;
scanf("%d%d",&n,&m);
for(i=0;i<m;i++)
scanf("%d%d%d",&e[i].u,&e[i].v,&e[i].w);
for(i=0;i<n;i++)parent[i]=i;
for(i=0;i<m-1;i++){
for(j=0;j<m-i-1;j++){
if(e[j].w>e[j+1].w){
temp=e[j];
e[j]=e[j+1];
e[j+1]=temp;
}
}
}
for(i=0;i<m;i++){
a=find(e[i].u);
b=find(e[i].v);
if(a!=b){
parent[a]=b;
printf("%d-%d:%d\n",e[i].u,e[i].v,e[i].w);
cost+=e[i].w;
```

```
count++;
}
}
if(count!=n-1)printf("No spanning tree\n");
else printf("Minimum cost=%d\n",cost);
return 0;
}
```

OUTPUT:

```
2-3:4
0-3:5
0-1:10
Minimum cost=19
```

4.Dijikstra’s algorithm.

```
#include<stdio.h>
#define MAX 100
#define INF 999999
int main(){
int g[MAX][MAX],d[MAX],vis[MAX]={0};
int n,src,i,j,u,min;
scanf("%d",&n);
for(i=0;i<n;i++)
for(j=0;j<n;j++){
scanf("%d",&g[i][j]);
if(i!=j&&g[i][j]==0)g[i][j]=INF;
}
scanf("%d",&src);
```

```
for(i=0;i<n;i++)d[i]=INF;
d[src]=0;
for(i=0;i<n;i++){
u=-1;
min=INF;
for(j=0;j<n;j++)
if(!vis[j]&&d[j]<min){min=d[j];u=j;}
if(u==-1)break;
vis[u]=1;
for(j=0;j<n;j++)
if(!vis[j]&&g[u][j]!=INF&&d[u]+g[u][j]<d[j])
d[j]=d[u]+g[u][j];
}
for(i=0;i<n;i++){
if(d[i]==INF)printf("%d:Unreachable\n",i);
else printf("%d:%d\n",i,d[i]);
}
return 0;
}
```

5.DAG for topological sorting.

```
#include<stdio.h>
#define MAX 100
int main(){
int g[MAX][MAX],indegree[MAX]={0},q[MAX];
int n,i,j,front=0,rear=0,count=0,v;
scanf("%d",&n);
```

```
for(i=0;i<n;i++)
for(j=0;j<n;j++){
scanf("%d",&g[i][j]);
if(g[i][j])indegree[j]++;
}
for(i=0;i<n;i++)
if(indegree[i]==0)q[rear++]=i;
while(front<rear){
v=q[front++];
printf("%d ",v);
count++;
for(j=0;j<n;j++){
if(g[v][j]){
indegree[j]--;
if(indegree[j]==0)q[rear++]=j;
}
}
}
if(count!=n)printf("\nGraph contains a cycle");
return 0;
}
```

6.Insert 76,93,40,47,10,55 in a hash table of size 7 using linear probing method .

```
#include<stdio.h>
#define SIZE 7
int main(){
int h[SIZE],a[]={76,93,40,47,10,55};
```

```
int i,j,pos;
for(i=0;i<SIZE;i++)h[i]=-1;
for(i=0;i<6;i++){
pos=a[i]%SIZE;
while(h[pos]!=-1)pos=(pos+1)%SIZE;
h[pos]=a[i];
}
for(i=0;i<SIZE;i++)printf("%d:%d\n",i,h[i]);
return 0;
}
```

7.Consider a hash table of size 7 ,hash function as hash (x)= a mod 7 and collision resolution strategy to be f(i)=i^2.Insert 22,30 and 50.

```
#include<stdio.h>
#define SIZE 7
int main(){
int h[SIZE],a[]={22,30,50};
int i,j,pos;
for(i=0;i<SIZE;i++)h[i]=-1;
for(i=0;i<3;i++){
for(j=0;j<SIZE;j++){
pos=(a[i]%SIZE+j*j)%SIZE;
if(h[pos]==-1){
h[pos]=a[i];
break;
}
}
```

```
if(j==SIZE)printf("Insertion failed for %d\n",a[i]);
}
for(i=0;i<SIZE;i++)printf("%d:%d\n",i,h[i]);
return 0;
}
```

8.Inseert the keys 27,43,692,72 into hash table of size 7 ,where first hash function id=s h(k)= k mod 7 and second hash function is h2(k)=1+(k mod 5) using double hashing technique.

```
#include<stdio.h>
#define SIZE 7
int main(){
int h[SIZE],a[]={27,43,692,72};
int i,j,pos,h1,h2;
for(i=0;i<SIZE;i++)h[i]=-1;
for(i=0;i<4;i++){
h1=a[i]%SIZE;
h2=1+(a[i]%5);
for(j=0;j<SIZE;j++){
pos=(h1+j*h2)%SIZE;
if(h[pos]==-1){
h[pos]=a[i];
break;
}
}
if(j==SIZE)printf("Insertion failed\n");
}
for(i=0;i<SIZE;i++)printf("%d:%d\n",i,h[i]);
```

return 0;

}

9.If quadratic probing is used and the table size is prime ,then a new element can always be inserted if the table is atleast half empty- prove the above statement.

Statement: If the table size p is prime and quadratic probing uses $h_i(k)=(h(k)+i^2)\bmod p$, a new element can be inserted whenever at least half of the table is empty.

Proof:

1. Let the table size be a prime number p.

2. Quadratic probing examines positions of the form

$$
(h(k)+i^2)\bmod p
$$

3. The values $i^2\bmod p$ produce $(p+1)/2$ distinct positions when i ranges from 0 to $p-1$. This follows because $i^2\equiv j^2\pmod p$ implies $i\equiv j$ or $i\equiv-j\pmod p$.

4. If at least half the table is empty, fewer than $(p+1)/2$ positions are occupied.

5. Since the probing sequence examines $(p+1)/2$ distinct positions, at least one of those positions must be empty.

Therefore, a new element can always be inserted. Hence proved.

10.Implement a hash data structure and write initialize ,find and insert routine hash table creation.

```
#include<stdio.h>

#define SIZE 10

int h[SIZE];

void initialize(){

int i;
```

```
for(i=0;i<SIZE;i++)h[i]=-1;
}
int find(int key){
int i,pos=key%SIZE;
for(i=0;i<SIZE;i++){
if(h[pos]==-1)return -1;
if(h[pos]==key)return pos;
pos=(pos+1)%SIZE;
}
return -1;
}
void insert(int key){
int i,pos;
if(find(key)!=-1){
printf("Duplicate key\n");
return;
}
pos=key%SIZE;
for(i=0;i<SIZE;i++){
if(h[pos]==-1){
h[pos]=key;
return;
}
pos=(pos+1)%SIZE;
}
printf("Hash table full\n");
```

```
}
void display(){
int i;
for(i=0;i<SIZE;i++)printf("%d:%d\n",i,h[i]);
}
int main(){
initialize();
insert(25);
insert(35);
insert(45);
display();
printf("Position: %d\n",find(35));
return 0;
}
```

OUTPUT:

```
0:-1
1:-1
2:-1
3:-1
4:35
5:45
6:-1
7:-1
8:-1
9:25
Position: 4
```